

Terminal

para Devs

Seu primeiro passo no mundo da tecnologia

Do zero ao terminal

3 Exercícios Práticos

Cheat Sheet

Sumário

01	O que é o terminal?	2
02	Abrindo o terminal	3
03	Navegando entre pastas	4
04	Gerenciando arquivos	5
05	Comandos essenciais	6
06	Exercício 1. Explorando o sistema [Iniciante]	8
07	Exercício 2. Criando seu projeto [Intermediário]	9
08	Exercício 3. Script do dia a dia [Desafio]	11
09	Cheat Sheet	13
10	Próximos passos	14

Como usar este mini book

Leia do começo ao fim na primeira vez.

Os capítulos foram pensados em sequência: cada um prepara o terreno para o seguinte. Se você pular direto para os exercícios sem ler os conceitos, pode ficar perdida em alguns comandos.

Volte sempre que precisar.

O Cheat Sheet da página 13 foi feito para ficar aberto enquanto você pratica. Deixe ele no segundo monitor ou impresso do lado do computador.

Faca os exercícios de verdade.

Ler sobre Docker ensina a teoria. Rodar os comandos é que fixa de verdade. Abra o terminal e execute cada passo. Não tem atalho aqui.

01 • O que é o terminal?

Quando você usa um computador, está acostumada a clicar em ícones, arrastar arquivos e usar menus bonitos. Isso é a interface gráfica. O terminal é outra forma de conversar com o computador: em vez de clicar, você **digita comandos**. Parece intimidador no começo, mas é mais simples do que parece.

O terminal é uma janela onde você digita instruções e o computador responde. É direto, rápido e extremamente poderoso. Toda dev que trabalha com backend, cloud ou DevOps usa o terminal todos os dias.

>> Pensa assim: a interface gráfica é como pedir um café no balcão apontando para a vitrine. O terminal é como ligar direto para a cozinha e falar exatamente o que você quer. Mais trabalhoso? Um pouco. Mais preciso e eficiente? Com certeza.

[DICA] Por que aprender o terminal?

Sem o terminal você não consegue usar o Docker, fazer deploy, configurar servidores na nuvem, ou trabalhar com Git pela linha de comando. É o pré-requisito número 1 de qualquer carreira em tecnologia.

02 · Abrindo o terminal

O primeiro passo é simplesmente abrir o terminal. O processo é diferente em cada sistema operacional. Escolha o seu:

Windows

A Microsoft criou o **WSL 2** (Windows Subsystem for Linux) que permite usar um terminal Linux direto no Windows. É a forma mais recomendada para devs.

- Pesquise por **Microsoft Store** no Windows
- Instale o **Ubuntu** (a versão mais recente disponível)
- Após instalar, abra o Ubuntu pelo menu Iniciar
- Na primeira vez, crie um usuário e senha

[DICA] Alternativa no Windows

Você também pode usar o **Git Bash** (que vem com o Git) ou o **PowerShell**. Mas o WSL 2 com Ubuntu é a opção mais completa e mais parecida com o ambiente de produção real.

Mac

O Mac já vem com um terminal Unix nativo. Você não precisa instalar nada.

- Pressione **Command + Espaço** para abrir o Spotlight
- Digite **Terminal** e pressione Enter
- Ou acesse: Aplicativos Utilitários Terminal

Linux

Se você já usa Linux, parabéns! Você já está em casa. O terminal varia por distribuição:

- Ubuntu/Debian: pressione **Ctrl + Alt + T**
- Ou pesquise por **Terminal** no menu de aplicativos

[DICA] Seu primeiro comando

Com o terminal aberto, digite o comando abaixo e pressione Enter. Ele mostra em qual pasta você está agora:

Terminal

```
pwd
```

```
# Saída esperada: /home/seuusuario ou /Users/seuusuario
```

>> O terminal é como uma conversa por WhatsApp com o seu computador. Você manda uma mensagem (o comando), ele responde. Às vezes a resposta é longa, às vezes é só um OK silencioso. E se errar, ele te avisa.

03 • Navegando entre pastas

No terminal, você está sempre **dentro de uma pasta**. Aprender a navegar entre pastas é o movimento mais fundamental que existe. São apenas 3 comandos que você vai usar todos os dias:

pwd — onde você está

Mostra o caminho completo da pasta atual. Use sempre que estiver perdida.

```
pwd
```

```
# Exemplo de saída:
```

```
# /home/gabi/projetos
```

>> É como olhar para o endereço na barra do Google Maps. Você sempre pode checar onde está antes de se mover.

ls — o que tem aqui

Lista todos os arquivos e pastas dentro da pasta atual.

```
ls          # lista básica
```

```
ls -la      # lista detalhada com arquivos ocultos
```

```
ls -lh      # lista com tamanho em formato legível
```

>> Abrir o explorador de arquivos e ver tudo que tem na pasta. O ls faz exatamente isso, só que no terminal.

cd — mudar de pasta

O comando mais usado. CD vem de *Change Directory*.

```
cd projetos          # entra na pasta projetos
cd ..                # volta uma pasta acima
cd ../outro-lugar    # volta e entra em outro lugar
cd ~                 # vai direto para a pasta home
cd /                  # vai para a raiz do sistema
```

>> É como clicar em uma pasta no explorador de arquivos. O 'cd ..' é como clicar na setinha de voltar.

[DICA] Dica de ouro: Tab para autocompletar

Você não precisa digitar o nome completo das pastas! Digite as primeiras letras e pressione **Tab**. O terminal completa automaticamente. Se tiver mais de uma opção, pressione Tab duas vezes para ver todas.

04 · Gerenciando arquivos

Agora que você sabe navegar, é hora de criar, copiar, mover e deletar arquivos e pastas pelo terminal. Esses comandos são o dia a dia de qualquer dev.

Criando arquivos e pastas

```
mkdir minha-pasta    # cria uma pasta
mkdir -p a/b/c        # cria pastas aninhadas de uma vez
touch arquivo.txt     # cria um arquivo vazio
echo "Olá!" > arquivo.txt # cria arquivo com conteúdo
```

>> *mkdir* é criar uma nova pastinha no seu armário. *touch* é pegar um post-it em branco e colocar na mesa.

Copiando e movendo

```
cp arquivo.txt copia.txt      # copia um arquivo
cp -r pasta/ outra-pasta/     # copia uma pasta inteira
mv arquivo.txt nova-pasta/    # move um arquivo
mv nome-antigo.txt nome-novo.txt # renomeia um arquivo
```

>> *cp* é como tirar xerox de um documento. *mv* é como pegar o documento original e colocar em outra gaveta, ou simplesmente renomear a etiqueta.

Deletando

```
rm arquivo.txt          # deleta um arquivo
rm -r pasta/            # deleta uma pasta e tudo dentro
rm -rf pasta/           # deleta sem pedir confirmação
```

[!] Cuidado com o rm -rf

O `rm -rf` deleta tudo permanentemente, sem passar pela lixeira. Não tem desfazer. Sempre verifique o que você está deletando antes de executar esse comando.

Lendo arquivos

```
cat arquivo.txt          # mostra o conteúdo inteiro
less arquivo.txt         # mostra página a página (q para sair)
head -5 arquivo.txt      # mostra as primeiras 5 linhas
tail -5 arquivo.txt      # mostra as últimas 5 linhas
tail -f log.txt          # acompanha o arquivo em tempo real
```

>> O `cat` é como abrir um documento e ler do começo ao fim. O `tail -f` é como ficar olhando para o Instagram em tempo real esperando nova notificação. Perfeito para acompanhar logs de aplicação rodando.

Buscando dentro de arquivos

O `grep` é um dos comandos mais poderosos do terminal. Ele busca texto dentro de arquivos.

```
grep 'erro' log.txt      # busca linhas com a palavra 'erro'
grep -i 'erro' log.txt   # busca sem diferenciar maiúsculas
grep -r 'def ' src/      # busca em todos os arquivos da pasta
grep -n 'import' app.py  # mostra o número da linha também
cat log.txt | grep 'ERROR' # filtra saída de outro comando
```

[DICA] O pipe |

O símbolo `|` (pipe) conecta a saída de um comando à entrada de outro. É como uma correia transportadora: o resultado do primeiro vira o ingrediente do segundo. Você vai ver isso em todos os projetos reais.

05 · Comandos essenciais

Além de navegar e gerenciar arquivos, o terminal tem uma série de comandos que você vai encontrar no dia a dia do trabalho como dev. Aqui estão os mais importantes:

Permissões com chmod

No Linux, cada arquivo tem permissões de leitura, escrita e execução. O chmod controla quem pode fazer o quê.

```
chmod +x script.sh      # torna o arquivo executável
chmod 755 script.sh     # permissões padrão para scripts
ls -la                  # ver as permissões de cada arquivo
```

>> É como definir quem pode abrir, editar ou executar um documento no Google Drive. Só que aqui é para arquivos no seu computador.

Variáveis de ambiente

Variáveis de ambiente são informações que o sistema disponibiliza para todos os programas. São muito usadas para guardar senhas, URLs e configurações.

```
export NOME=Gabi        # cria uma variável
echo "Olá, $NOME"       # usa a variável
env                      # lista todas as variáveis
echo $PATH              # variável especial com caminhos do sistema
```

>> É como o caderno de anotações global do sistema. Qualquer programa pode ler essas informações. Por isso nunca colocamos senhas diretamente no código, mas sim em variáveis de ambiente.

Processos e histórico

```
ps aux                  # lista todos os processos rodando
kill 1234               # encerra um processo pelo ID
ctrl + C               # interrompe o processo atual
ctrl + Z               # pausa o processo atual
history                 # mostra os últimos comandos usados
!!                     # repete o último comando
ctrl + R               # busca no histórico de comandos
```

Instalando pacotes

No Linux você instala programas pelo terminal, sem precisar de um instalador gráfico.

Ubuntu/Debian

```
sudo apt-get update           # atualiza a lista de pacotes
sudo apt-get install nome     # instala um programa
sudo apt-get remove nome      # desinstala um programa
sudo apt-get upgrade          # atualiza todos os programas
```

Atalhos que salvam tempo

Atalho	O que faz
Tab	Autocompleta o comando ou nome de arquivo
Ctrl + C	Cancela o comando em execução
Ctrl + L	Limpa a tela do terminal
Ctrl + R	Busca no histórico de comandos
Seta ↑ ↓	Navega pelos comandos anteriores
Ctrl + A	Move o cursor para o início da linha
Ctrl + E	Move o cursor para o final da linha

Exercício 1 — Explorando o sistema

INICIANTE

Objetivo: Abrir o terminal e praticar navegação e listagem de arquivos com confiança.

Tempo estimado: 15 minutos

O que você vai aprender

- Como abrir o terminal e se sentir em casa nele
- Como descobrir onde você está com **pwd**
- Como listar arquivos e pastas com **ls**
- Como navegar entre pastas com **cd**

Passo a passo

1. Abra o terminal e descubra onde você está:

```
pwd
# Anote o caminho que apareceu. Essa é sua home.
```

2. Liste o que tem na sua pasta atual:

```
ls
ls -la    # versão detalhada com arquivos ocultos
```

3. Navegue pelas pastas do sistema e explore:

```
cd Documents    # ou cd Documentos no sistema em PT
pwd             # confirme onde você está
ls              # veja o que tem aqui
cd ..           # volte uma pasta
cd ~            # vá para a home de qualquer lugar
```

4. Pratique o Tab para autocompletar:

```
# Digite 'cd Do' e pressione Tab
# O terminal completa automaticamente
# Se tiver mais de uma opção, pressione Tab duas vezes
```

[OK] Resultado esperado

Você abriu o terminal sem pânico, navegou por pelo menos 3 pastas diferentes e usou o Tab para autocompletar pelo menos uma vez. Isso já é mais do que muita gente sabe fazer!

[OK] Resultado esperado — Exercício 1

Você abriu o terminal, navegou por pastas e usou o Tab. Isso já é mais do que muita gente sabe fazer!

Exercício 2 — Criando seu projeto

INTERMEDIÁRIO

Objetivo: Criar uma estrutura de projeto completa usando apenas o terminal.

Tempo estimado: 20 minutos

O que você vai aprender

- Como criar pastas e arquivos pelo terminal
- Como escrever conteúdo em arquivos com **echo**
- Como ler arquivos com **cat**
- Como copiar, mover e renomear arquivos

Vamos criar do zero a estrutura de um projeto real, como você faria no trabalho.

Passo a passo

1. Crie a estrutura de pastas do projeto:

```
cd ~
mkdir meu-primeiro-projeto
cd meu-primeiro-projeto
mkdir src tests docs
ls      # confirme que as 3 pastas foram criadas
```

2. Crie os arquivos do projeto:

```
touch src/app.py
touch tests/test_app.py
echo "# Meu Primeiro Projeto" > README.md
echo "# dependências do projeto" > requirements.txt
```

3. Verifique a estrutura criada:

```
ls -la
ls -la src/
cat README.md      # leia o conteúdo do README
```

4. Pratique copiar e mover:

```
cp README.md docs/README.md
mv requirements.txt src/requirements.txt
ls src/      # confirme que ambos estão lá
```

>> *Você acabou de montar um closet organizado do zero: seção de roupas (src), seção de testes (tests) e seção de documentos (docs). Tudo no lugar, tudo organizado, sem nunca abrir o explorador de arquivos.*

[OK] Resultado esperado

Você criou uma estrutura real de projeto com 3 pastas e 4 arquivos usando só o terminal. Se conseguiu ver o conteúdo do README com cat — exercício concluído com sucesso!

Bônus: visualize a estrutura completa

O comando tree mostra a estrutura de pastas de forma visual. Se não estiver instalado, instale com `sudo apt-get install tree`:

```
tree meu-primeiro-projeto/
# Saída esperada:
# meu-primeiro-projeto/
# |— README.md
# |— docs/
# |   |— README.md
# |— src/
# |   |— app.py
# |   |— requirements.txt
# |— tests/
#     |— test_app.py
```

[OK] Resultado esperado — Exercício 2

Você criou uma estrutura real de projeto com 3 pastas e 4 arquivos usando só o terminal.

Exercício 3 — Script do dia a dia

DESAFIO

Objetivo: Criar um script shell que automatiza uma tarefa real do seu dia a dia como dev.

Tempo estimado: 30 minutos

[DICA] O que é um script shell?

Um script shell é um arquivo de texto com uma lista de comandos do terminal que o computador executa em sequência. É como criar uma receita de bolo: você escreve os passos uma vez e pode repetir quantas vezes quiser com um único comando.

O que você vai aprender

- Como criar e executar um script shell
- Como usar variáveis e condições básicas
- Como usar **chmod** para tornar o script executável
- Como automatizar uma tarefa repetitiva real

Passo a passo

1. Crie o arquivo do script:

```
cd ~  
touch organizar-projeto.sh  
chmod +x organizar-projeto.sh
```

2. Abra o arquivo para editar com o nano:

```
nano organizar-projeto.sh  
# O nano é um editor de texto simples no terminal  
# Para salvar: Ctrl + O, depois Enter  
# Para sair: Ctrl + X
```

3. Digite o conteúdo do script:

```
organizar-projeto.sh
#!/bin/bash
# Script para criar estrutura de projeto
PROJETO=$1 # recebe o nome como argumento
if [ -z "$PROJETO" ]; then
    echo "Uso: ./organizar-projeto.sh nome-do-projeto"
    exit 1
fi
echo "Criando projeto: $PROJETO"
mkdir -p $PROJETO/{src,tests,docs}
touch $PROJETO/README.md
touch $PROJETO/src/app.py
touch $PROJETO/tests/test_app.py
echo "# $PROJETO" > $PROJETO/README.md
echo "Projeto $PROJETO criado com sucesso!"
ls -la $PROJETO/
```

4. Execute o script com o nome do seu projeto:

```
./organizar-projeto.sh minha-api
# O script vai criar toda a estrutura automaticamente!
./organizar-projeto.sh outro-projeto
# Execute de novo com outro nome para ver a mágica
```

>> Você criou uma receita automática. Antes, montar a estrutura de um projeto levava vários comandos. Agora é um único comando e o computador faz tudo. É assim que devs economizam tempo todos os dias.

[OK] Resultado esperado

O script executa sem erros e cria a estrutura completa do projeto automaticamente. Se você conseguiu rodar duas vezes com nomes diferentes e ver as pastas sendo criadas, você acabou de escrever seu primeiro programa em shell!

Erros comuns e como resolver

- **Permission denied:** você esqueceu do `chmod +x`. Execute `chmod +x organizar-projeto.sh` e tente novamente.
- **command not found:** certifique-se de executar com `./` na frente:
`./organizar-projeto.sh`
- **bash: syntax error:** verifique se copiou o script exatamente como está, especialmente os espaços dentro dos colchetes `[ -z ]`.

- **nano não abre:** tente usar o vi ou o gedit dependendo do seu sistema.

[DICA] Expandindo o script

Agora que você sabe o básico, experimente expandir o script. Tente adicionar uma mensagem de boas-vindas com a data de criação do projeto usando o comando `date`, ou criar um arquivo `.gitignore` automaticamente. Cada pequena automação que você cria economiza minutos que viram horas no longo prazo.

Próximo nível: scripts mais poderosos

organizar-projeto.sh (versão expandida)

```
#!/bin/bash
# Mostra a data de criação no README
DATA=$(date "+%d/%m/%Y")
echo "Criado em: $DATA" >> $PROJETO/README.md
# Cria .gitignore automaticamente
cat > $PROJETO/.gitignore << 'EOF'
__pycache__/
*.pyc
.env
venv/
EOF
echo ".gitignore criado!"
```

09 • Cheat Sheet de Comandos

Cola essa página nos favoritos. São os comandos do terminal que você vai usar toda semana como dev:

Navegação	
<code>pwd</code>	Mostra a pasta atual
<code>ls</code>	Lista arquivos e pastas
<code>ls -la</code>	Lista com detalhes e ocultos
<code>cd pasta</code>	Entra em uma pasta
<code>cd ..</code>	Volta uma pasta
<code>cd ~</code>	Vai para a home
Arquivos e Pastas	
<code>mkdir nome</code>	Cria uma pasta
<code>mkdir -p a/b/c</code>	Cria pastas aninhadas
<code>touch arquivo.txt</code>	Cria um arquivo vazio
<code>cp origem destino</code>	Copia um arquivo
<code>mv origem destino</code>	Move ou renomeia
<code>rm arquivo</code>	Deleta um arquivo
<code>rm -rf pasta/</code>	Deleta uma pasta inteira
Leitura e Busca	
<code>cat arquivo</code>	Mostra o conteúdo
<code>less arquivo</code>	Mostra página a página
<code>head -n arquivo</code>	Primeiras n linhas
<code>tail -f arquivo</code>	Acompanha em tempo real
<code>grep texto arquivo</code>	Busca texto em arquivo
<code>find . -name '*.py'</code>	Busca arquivos por nome

Sistema e Processos	
chmod +x script.sh	Torna arquivo executável
sudo comando	Executa como administradora
ps aux	Lista processos rodando
kill PID	Encerra um processo
history	Histórico de comandos
ctrl + C	Cancela o comando atual
ctrl + L	Limpa a tela

10 • Próximos Passos

Parabéns por chegar até aqui! O terminal já não é mais um mistério para você. A série continua com temas que dependem direto do que você aprendeu agora:

Vol. 02 — Git e GitHub

Agora que você sabe usar o terminal, vem aprender a controlar as versões do seu código. O Git é usado via terminal e é habilidade obrigatória para qualquer dev.

Vol. 03 — Docker para Devs

Com terminal dominado, o Docker faz muito mais sentido. Todos os comandos do Docker rodam direto no terminal que você acabou de aprender.

Vol. 04 — Docker Compose

O próximo passo natural após o Docker. Orquestre múltiplos containers com um único arquivo de configuração.

Recursos para continuar aprendendo

- **The Linux Command Line** de William Shotts — livro gratuito online, o mais completo sobre terminal
- **explainshell.com** — cole qualquer comando e ele explica cada parte
- **tldr.sh** — versões simplificadas das páginas de manual dos comandos

- **overthewire.org/wargames/bandit** — aprenda terminal jogando, gratuito e viciante

O que você dominou neste volume

- O que é o terminal e por que toda dev precisa saber usá-lo
- Como navegar pelo sistema com pwd, ls e cd
- Como criar, copiar, mover e deletar arquivos e pastas
- Como ler arquivos e acompanhar logs em tempo real
- Como usar permissões e variáveis de ambiente
- Como criar e executar seu primeiro script shell

Você chegou até aqui. Isso já te coloca na frente de muita gente.

O terminal não é só uma ferramenta. É a linguagem que o computador entende de verdade.

Gostou desse mini book?

Compartilha com aquela amiga dev que tá começando a jornada!

Me marca no Instagram **@deployada** com o seu progresso.

Adoro acompanhar vocês evoluindo!

Instagram	LinkedIn	Newsletter
@deployada	linkedin.com/in/mgcarvalho	deployada

Continue sua jornada! >>

Vol. 01 • Docker Compose. orquestrando containers

Vol. 05 • APIs REST com Python

Vol. 06 • AWS na prática: S3, EC2 e Lambda

Vol. 07 • Mensageria: SQS & SNS

Vol. 08 • Observabilidade: Datadog & Splunk

"Toda grande dev um dia rodou seu primeiro container."

— @deployada

@deployada

Instagram • LinkedIn • Newsletter